Department of Electrical, Computer and Biomedical Engineering
Toronto Metropolitan University
COE 865 Project Assignment
Design and Implementation of Multicast Overlay Network

Due Date: April 5, 2026

A Multicast Overlay Network (MON) is a connected graph of nodes and links. Nodes are hosts that are connected to a network, and links are transport-level connections, e.g. TCP or UDP connections, between the two nodes. In a multicast overlay network a set of nodes are deployed as multicast nodes. Each node acts as a *forwarder* to forward the multicast packet. Some nodes are also configured as *receivers* and as *senders.* A node is connected to one or more multicast nodes through TCP or UPD protocol. For example, node *1* in Figure 1 is the multicast sender that sends multicast packets to nodes 4, 6 and 7 that are receivers; and nodes 2, 3 and 5 only forward those packets. You may use link costs, as shown in the figure, in your routing heuristic.

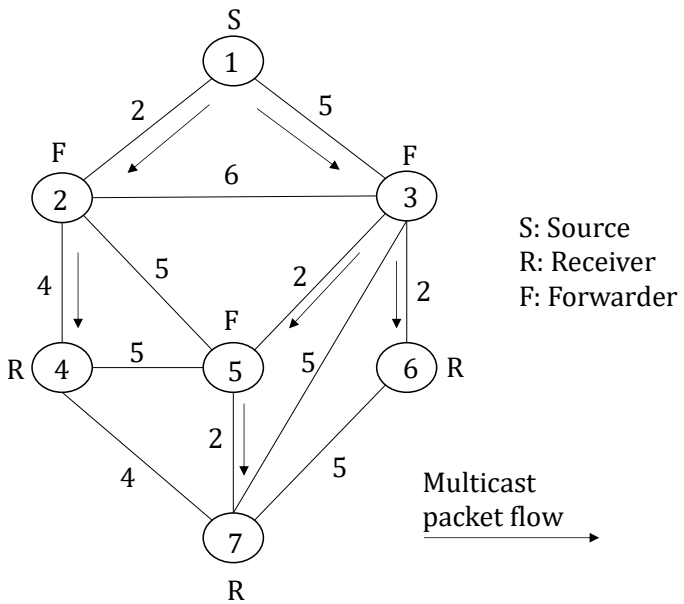

Figure 1: A Multicast Overlay Network

In this project, you need to do the following:

1. Design and implement multicast routing that builds multicast distribution tree at each node and maintains the next hop for each (group, receiver) pair. For example, you can use Biased Random Walk of GIA to search for all the receivers. You may assume building a single tree for one (S, G). In your report, you can explain how to build multiple trees.
2. Write a program to implement a simple version of the above Multicast Overlay Network for one multicast group with your distributed routing algorithm in Python (or C, C++, Java). The recommended programming language is Python using Python 3.10 or higher version.

Program

- You can also use socket APIs to implement communication routines. You can visit https://realpython.com/python-sockets/#toc (or http://beej.us/guide/bgnet/) for on-line tutorials on socket programming.
- You can use loopback address, 127.0.0.1, and port number within the following interval (65000 < port numbers < 65535) at each node to run all of them on the same computer.
- You can either use TCP (preferrable for reliability reason) or UDP socket to establish the overlay connection with the peers for exchanging messages.
- **Initialization Phase:** During the bootstrap process, every node reads its configuration file and opens connections with peer nodes. You can design your own bootstrap process that may change the structure of the *config* file.
- State all your assumptions clearly in the report.
- You should submit the code tested for at least one test topology.
- **Input:** The input to the program is a *config* file.
- **Output:** Display on the screen the content of every message a node receives and generates in a readable format. Display the routing table at each node.
- **Submission:** *You can work in a group of no more than two students*. You should submit a tar file containing the source code and necessary configuration files [e.g. *.py*, *.c* or *.java* files, *makefile*, *config* files], test run script, and the report. The report must describe your design, all the program files, major algorithms, message structures, test cases and the topologies you use for testing your program. You should also include analysis of your routing algorithm. You should also identify in the report the contribution of each group member. *Report will be evaluated not only on the technical content but also on the quality of presentation. You should not use AI tools to generate the report. Identify any tools you use for grammar check.*

Marking

Project Demo	25%
Program (code)	25%
Report	50%